

Министерство науки и высшего образования Российской Федерации  
Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
«Алтайский государственный технический университет им. И.И. Ползунова»  
Университетский технологический колледж  
Кафедра Информационные системы в экономике

Отчет защищен с оценкой \_\_\_\_\_  
Преподаватель \_\_\_\_\_ С.В. Умбетов  
«    » \_\_\_\_\_ 2026 г.

Отчет по лабораторной работе №2  
по дисциплине «Инженерно-техническая поддержка сопровождения  
интеллектуальных информационных систем» «Основы работы с GIT»

ЛР 09.02.08.23.002

|                |                                      |                      |
|----------------|--------------------------------------|----------------------|
| Студент группы | <u>1ИИС-41</u>                       | <u>М.А. Ситников</u> |
|                | группа                               | фи.о., фамилия       |
| Преподаватель  | <u>старший преподаватель, доцент</u> | <u>С.В. Умбетов</u>  |
|                | должность, ученая степень            | фи.о., фамилия       |

Барнаул 2026

## **Лабораторная работа №2**

### **Основы работы с GIT**

**Цели и задачи работы:** Целью лабораторной работы является освоение базовых и продвинутых приёмов работы с распределённой системой контроля версий Git в операционной системе Linux. В ходе выполнения работы предполагается изучить процесс установки и настройки Git, освоить основные команды для работы с локальным репозиторием, научиться управлять состояниями файлов, работать с ветками, выполнять слияния и разрешать конфликты. Кроме того, целью является формирование практических навыков организации версионного контроля при разработке программного обеспечения и понимания роли Git в командной работе.

**Задание к работе:** Выполнить задачи 1–20 в соответствии с условиями. Ответить на контрольные вопросы.

**Задание принято:**  
**Ситников М.А.**



Подпись

## Ход работы

### 1. Откройте терминал и проверьте наличие Git в системе командой `git --version`.

```
root@sitnikov1iis41-VirtualBox:~# git --version
Команда «git» не найдена, но может быть установлена с помощью:
apt install git
root@sitnikov1iis41-VirtualBox:~#
```

Рис.1 – Проверка наличия Git в системе командой `git --version`

**Результат:** Проверка показала, что git не установлен на данной ОС

### 2. Определите свой дистрибутив Linux командой `cat /etc/os-release`.

```
root@sitnikov1iis41-VirtualBox:~# cat /etc/os-release
PRETTY_NAME="Ubuntu 22.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
VERSION="22.04.5 LTS (Jammy Jellyfish)"
VERSION_CODENAME=jammy
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=jammy
root@sitnikov1iis41-VirtualBox:~# _
```

Рис.2 – Определение дистрибутива командой `cat /etc/os-release`

### 3. Установите Git с помощью менеджера пакетов дистрибутива.

```
root@sitnikov1iis41-VirtualBox:~# sudo apt-get install git
Чтение списков пакетов... Готово
Построение дерева зависимостей... Готово
Чтение информации о состоянии... Готово
Будут установлены следующие дополнительные пакеты:
  git-man liberror-perl
Предлагаемые пакеты:
  git-daemon-run | git-daemon-sysvinit git-doc git-email git-gui gitk gitweb git-cvs git-mediawiki git-svn
Следующие НОВЫЕ пакеты будут установлены:
  git git-man liberror-perl
Обновлено 0 пакетов, установлено 3 новых пакетов, для удаления отмечено 0 пакетов, и 460 пакетов не обновлено.
Необходимо скачать 4 154 kB архивов.
После данной операции объём занятого дискового пространства возрастёт на 21,0 MB.
Хотите продолжить? [Д/Н] Y
Пол:1 http://ru.archive.ubuntu.com/ubuntu jammy/main amd64 liberror-perl all 0.17029-1 [26,5 kB]
Пол:2 http://ru.archive.ubuntu.com/ubuntu jammy-updates/main amd64 git-man all 1:2.34.1-1ubuntu1.17 [954 kB]
Пол:3 http://ru.archive.ubuntu.com/ubuntu jammy-updates/main amd64 git amd64 1:2.34.1-1ubuntu1.17 [3 174 kB]
Получено 4 154 kB за 1с (4 900 kB/s)
Выбор ранее не выбранного пакета liberror-perl.
(Чтение базы данных ... на данный момент установлено 207803 файла и каталога.)
Подготовка к распаковке .../liberror-perl_0.17029-1_all.deb ...
Распаковывается liberror-perl (0.17029-1) ...
Выбор ранее не выбранного пакета git-man.
Подготовка к распаковке .../git-man_1%3a2.34.1-1ubuntu1.17_all.deb ...
Распаковывается git-man (1:2.34.1-1ubuntu1.17) ...
Выбор ранее не выбранного пакета git.
Подготовка к распаковке .../git_1%3a2.34.1-1ubuntu1.17_amd64.deb ...
Распаковывается git (1:2.34.1-1ubuntu1.17) ...
Настраивается пакет liberror-perl (0.17029-1) ...
Настраивается пакет git-man (1:2.34.1-1ubuntu1.17) ...
Настраивается пакет git (1:2.34.1-1ubuntu1.17) ...
Обрабатываются триггеры для man-db (2.10.2-1) ...
root@sitnikov1iis41-VirtualBox:~# _
```

Рис.3 – Установка Git командой `sudo apt-get install git`

**4. Повторно проверьте версию Git командой `git --version` для подтверждения успешной установки.**

```
root@sitnikov1iis41-VirtualBox:~# git --version
git version 2.34.1
root@sitnikov1iis41-VirtualBox:~#
```

Рис.4 – Повторная проверка наличия Git в системе

**5. Настройте имя пользователя командой `git config --global user.name "Ваше Имя"`.**

```
root@sitnikov1iis41-VirtualBox:~# git config --global user.name "Maksim Sitnikov"
root@sitnikov1iis41-VirtualBox:~#
```

Рис.5 – Настройка пользователя Maksim Sitnikov

**6. Настройте адрес электронной почты командой `git config --global user.email your.email@example.com`.**

```
root@sitnikov1iis41-VirtualBox:~# git config --global user.email maks.sitnikov.20@mail.ru
root@sitnikov1iis41-VirtualBox:~#
```

Рис.6 – Настройка электронной почты [gladysyielding@uberip.com](mailto:gladysyielding@uberip.com)

**7. Проверьте применённые настройки командой `git config --list`.**

```
root@sitnikov1iis41-VirtualBox:~# git config --list
user.name=Maksim Sitnikov
user.email=maks.sitnikov.20@mail.ru
root@sitnikov1iis41-VirtualBox:~#
```

Рис.7 – Настройки проверены с помощью команды `git config --list`

**8. Создайте рабочую директорию командой `mkdir git-practice` и перейдите в неё командой `cd git-practice`.**

```
root@sitnikov1iis41-VirtualBox:~# mkdir git-practice
root@sitnikov1iis41-VirtualBox:~# cd git-practice
root@sitnikov1iis41-VirtualBox:~/git-practice# _
```

Рис.8 – Рабочая директория создана командой `mkdir git-practice` и открыта командой `cd git-practice`

**9. Инициализируйте локальный репозиторий командой `git init`.**

```
root@sitnikov1iis41-VirtualBox:~/git-practice# git init
подсказка: Using 'master' as the name for the initial branch. This default branch name
подсказка: is subject to change. To configure the initial branch name to use in all
подсказка: of your new repositories, which will suppress this warning, call:
подсказка:
подсказка:     git config --global init.defaultBranch <name>
подсказка:
подсказка: Names commonly chosen instead of 'master' are 'main', 'trunk' and
подсказка: 'development'. The just-created branch can be renamed via this command:
подсказка:
подсказка:     git branch -m <name>
Инициализирован пустой репозиторий Git в /root/git-practice/.git/
root@sitnikov1iis41-VirtualBox:~/git-practice#
```

Рис.9 – Локальный репозиторий инициализирован командой `git init`.

## 10. Убедитесь в создании скрытой папки .git командой ls -la.

```
root@sitnikov1iis41-VirtualBox:~/git-practice# ls -la
итого 12
drwxr-xr-x 3 root root 4096 сен 29 13:27 .
drwx----- 6 root root 4096 сен 29 13:26 ..
drwxr-xr-x 7 root root 4096 сен 29 13:27 .git
root@sitnikov1iis41-VirtualBox:~/git-practice#
```

Рис.10 – Создание скрытой папки .git проверено командой ls -la

## 11. Создайте несколько файлов, например readme.txt, script.sh и data.json, с помощью команды touch.

```
root@sitnikov1iis41-VirtualBox:~/git-practice# touch readme.txt
root@sitnikov1iis41-VirtualBox:~/git-practice# touch script.sh
root@sitnikov1iis41-VirtualBox:~/git-practice# touch data.json
root@sitnikov1iis41-VirtualBox:~/git-practice#
```

Рис.11 – Файлы readme.txt, script.sh и data.json созданы с помощью команды touch

## 12. Проверьте статус репозитория командой git status и определите состояние созданных файлов (untracked).

```
root@sitnikov1iis41-VirtualBox:~/git-practice# git status
Текущая ветка: master

Еще нет коммитов

Неотслеживаемые файлы:
  (используйте «git add <файл>...», чтобы добавить в то, что будет включено в коммит)
    data.json
    readme.txt
    script.sh

индекс пуст, но есть неотслеживаемые файлы
(используйте «git add», чтобы проиндексировать их)
root@sitnikov1iis41-VirtualBox:~/git-practice# _
```

Рис.12 – Репозиторий проверен командой git status. Созданные файлы не отслеживаются в репозитории, поэтому они находятся в состоянии untracked

## 13. Добавьте в состояние staged только текстовые файлы командой git add \*.txt.

```
root@sitnikov1iis41-VirtualBox:~/git-practice# git add readme.txt
root@sitnikov1iis41-VirtualBox:~/git-practice# _
```

Рис.13 – Файл readme.txt добавлен в состояние staged

#### 14.Добавьте все остальные файлы командой `git add -A` и повторно проверьте статус командой `git status`.

```
root@sitnikov1iis41-VirtualBox:~/git-practice# git add -A
root@sitnikov1iis41-VirtualBox:~/git-practice# git status
Текущая ветка: master

Еще нет коммитов

Изменения, которые будут включены в коммит:
  (используйте «git rm --cached <файл>...», чтобы убрать из индекса)
    новый файл:   data.json
    новый файл:   readme.txt
    новый файл:   script.sh

root@sitnikov1iis41-VirtualBox:~/git-practice# _
```

Рис.14 – Все остальные файлы добавлены в состояние staged командой `git add -A`. Статус проверен командой `git status`

#### 15.Создайте первый коммит командой `git commit -m "initial commit: added base files"`.

```
root@sitnikov1iis41-VirtualBox:~/git-practice# git commit -m "initial commit: added base files"
[master (корневой коммит) dd41179] initial commit: added base files
 3 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 data.json
 create mode 100644 readme.txt
 create mode 100644 script.sh
root@sitnikov1iis41-VirtualBox:~/git-practice#
```

Рис.15 – Коммит создан командой `git commit -m "initial commit: added base files"`

#### 16.Просмотрите историю коммитов командой `git log` и зафиксируйте метаданные коммита.

```
root@sitnikov1iis41-VirtualBox:~/git-practice# git log
commit dd4117981819c4453e17d81759cb89dbfa2aa877 (HEAD -> master)
Author: Maksim Sitnikov <maks.sitnikov.20@mail.ru>
Date:   Tue Sep 29 13:38:41 2026 +0800

    initial commit: added base files
root@sitnikov1iis41-VirtualBox:~/git-practice#
```

Рис.16 – История коммитов просмотрена командой `git log` и зафиксирована

#### Зафиксированы метаданные коммита:

commit dd4117981819c4453e17d81759cb89dbfa2aa877 – идентификатор коммита

HEAD -> master – текущая версия

Author:Maksim Sitnikov <maks.sitnikov.20@mail.ru> - имя и почта автора

Date: Tue Sep 29 13:38:41 2026 +0800 – дата создания коммита

Initial commit: Added base files – комментарии к коммиту от автора

## 17. Внесите изменения в файл `readme.txt` и просмотрите различия командой `git diff`.

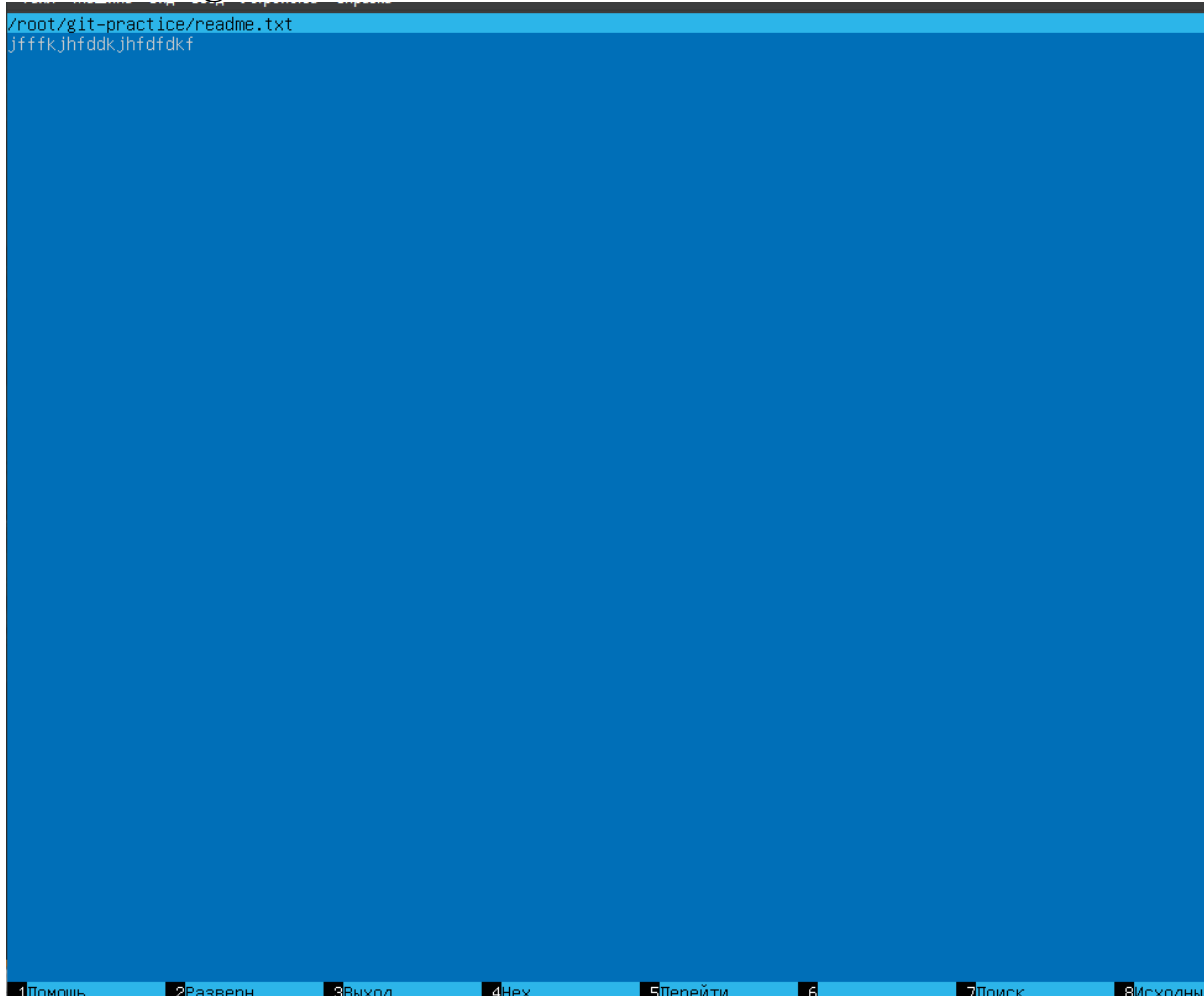


Рис.17 – В файл `readme.txt` введены изменения с помощью программы `mc`

```
root@sitnikov1iis41-VirtualBox:~/git-practice# git diff
diff --git a/readme.txt b/readme.txt
index e69de29..6e3f45e 100644
--- a/readme.txt
+++ b/readme.txt
@@ -0,0 +1 @@
+jffffkjhfddkjhfdfdkf
root@sitnikov1iis41-VirtualBox:~/git-practice# _
```

Рис.18 – Просмотр различий между файлами командой `git diff`

## 18. Создайте файл `.gitignore` командой `touch .gitignore` и внесите в него шаблоны `*.log`, `temp/`, `*.tmp`, `.cache/`.

```
root@sitnikov1iis41-VirtualBox:~/git-practice# touch .gitignore
root@sitnikov1iis41-VirtualBox:~/git-practice# _
```

Рис.19 – Файл `.gitignore` создан командой `touch .gitignore`

```
/root/git-practice/.gitignore
*.log
temp/
*.tmp
.cache/
```

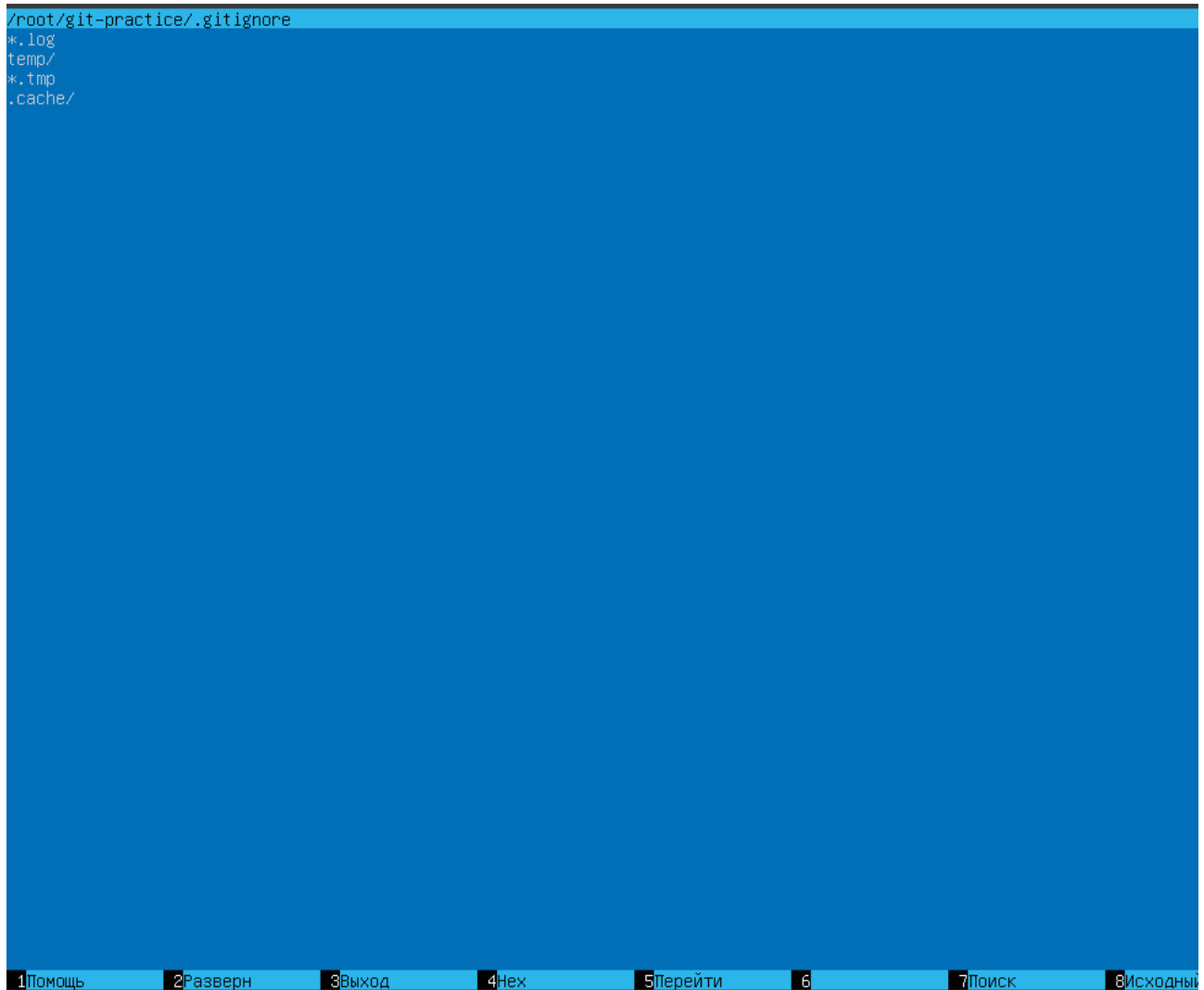


Рис.20 – В файл .gitignore внесены шаблоны .log, temp/, .tmp, .cache/ с помощью `mc`

## 19.Создайте новую ветку командой `git checkout -b feature/update-readme` и проверьте список веток командой `git branch -a`.

```
root@sitnikov1iis41-VirtualBox:~/git-practice# git checkout -b feature/update-readme
Переключились на новую ветку «feature/update-readme»
root@sitnikov1iis41-VirtualBox:~/git-practice# git branch -a
* feature/update-readme
  master
root@sitnikov1iis41-VirtualBox:~/git-practice# _
```

Рис.21 – Новая ветка создана командой `git checkout -b feature/update-readme` и проверена с помощью `git branch -a`

**20.Внесите изменения в файл readme.txt в новой ветке, закоммитьте их, переключитесь на master командой `git checkout master`, выполните слияние командой `git merge feature/update-readme` и проанализируйте итоговую историю командой `git log --oneline --graph --all`.**

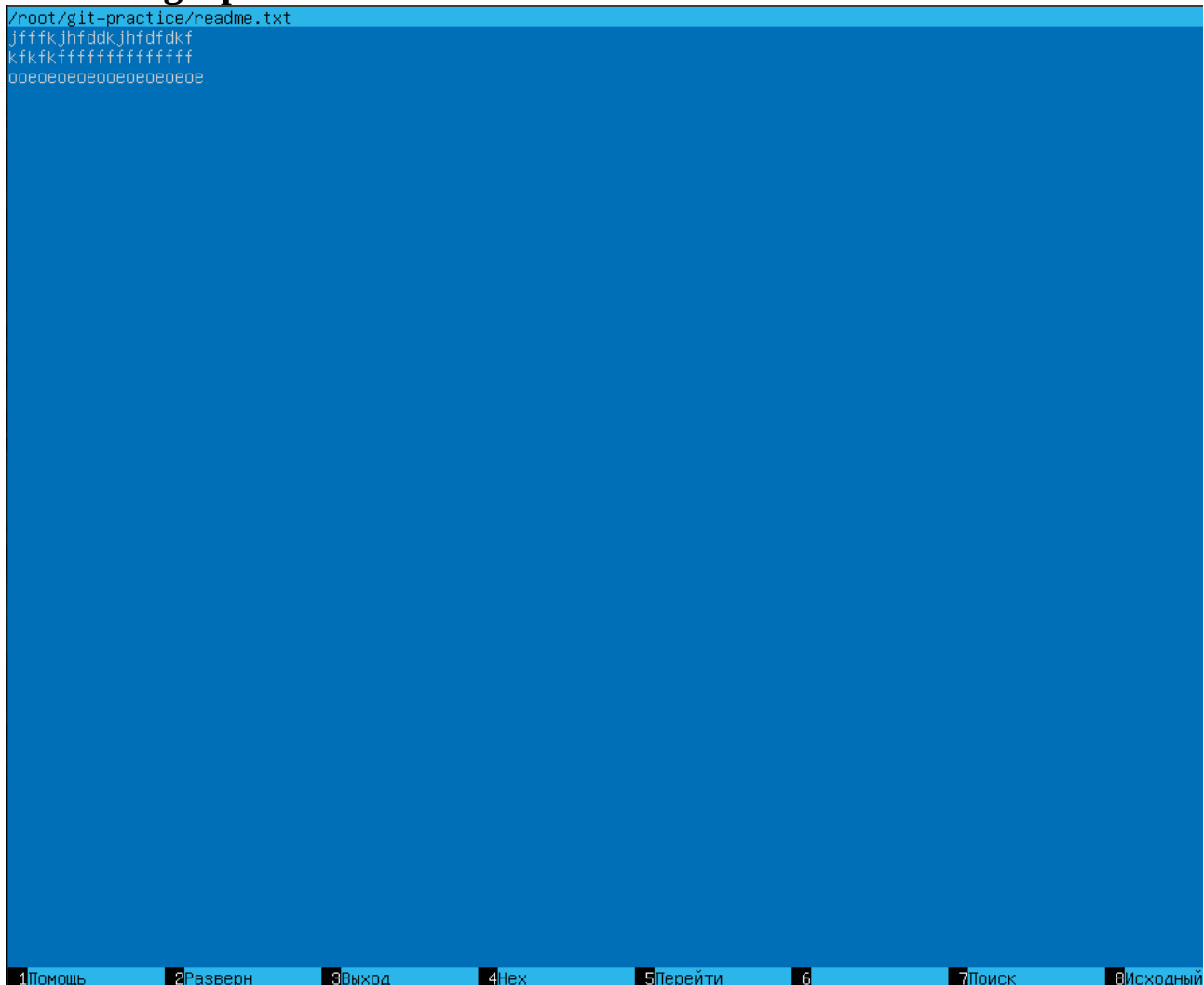


Рис.23 – В файл readme.txt введены изменения в новой ветке

```
root@sitnikov1iis41-VirtualBox:~/git-practice# git add readme.txt
root@sitnikov1iis41-VirtualBox:~/git-practice# git commit -m "readme updated"
[feature/update-readme f08de1a] readme updated
1 file changed, 3 insertions(+)
root@sitnikov1iis41-VirtualBox:~/git-practice# _
```

Рис.24 – Изменения в readme.txt закоммичены командой `git commit -m “readme updated”`

```
root@sitnikov1iis41-VirtualBox:~/git-practice# git checkout master
Переключились на ветку «master»
root@sitnikov1iis41-VirtualBox:~/git-practice# _
```

Рис.25 – Переключение на master командой `git checkout master`

```

root@sitnikov1iis41-VirtualBox:~/git-practice# git merge feature/update-readme
Обновление dd41179..f08de1a
Fast-forward
 README.txt | 3 +++
 1 file changed, 3 insertions(+)
root@sitnikov1iis41-VirtualBox:~/git-practice#

```

Рис.26 – Выполнение слияния командой `git merge feature/update-readme`

```

root@sitnikov1iis41-VirtualBox:~/git-practice# git log --oneline --graph --all
* f08de1a (HEAD -> master, feature/update-readme) README updated
* dd41179 initial commit: added base files
root@sitnikov1iis41-VirtualBox:~/git-practice#

```

Рис.27 – Итоговая история проанализирована командой `git log --oneline --graph --all`

## Ссылка на GitHub

[https://github.com/Enot2130/Sitnikov\\_1iis41/tree/main/Основы%20работы%20с%20GIT](https://github.com/Enot2130/Sitnikov_1iis41/tree/main/Основы%20работы%20с%20GIT)

## Контрольные вопросы

### 1. Что такое Git и для чего он используется?

Git — это распределённая система контроля версий, предназначенная для управления исходным кодом, отслеживания изменений, возврата к предыдущим состояниям и организации командной работы.

### 2. В чём разница между локальным и удалённым репозиторием?

Локальный репозиторий хранится на компьютере пользователя (в папке `.git`).

Удалённый репозиторий хранится на сервере, локально хранится только копия проекта, изменения в которую можно отправить командой `git push`.

### 3. Какая команда проверяет наличие Git в системе и его версию?

`git --version`.

### 4. Какие команды используются для установки Git в Ubuntu, Fedora и Arch Linux?

Ubuntu/Debian: `sudo apt-get install git`

Fedora: `sudo dnf install git`

Arch Linux: `sudo pacman -S git`

### 5. Для чего нужна настройка `user.name` и `user.email` в Git?

Для идентификации автора коммита. Git использует эти данные для заполнения поля `Author`.

### 6. В чём разница между командами `git config --global` и `git config` без флага?

С флагом `--global` настройки применяются для всех проектов пользователя на компьютере. Без флага — только для текущего конкретного проекта.

### 7. Что создаётся в директории при выполнении команды `git init`?

Создаётся скрытая папка `.git`, в которой хранится вся информация о репозитории.

### 8. Для чего предназначена папка `.git` и можно ли её удалять?

Папка `.git` хранит всю историю коммитов и служебную информацию. Удалять её нельзя, так как это приведёт к потере всей истории версий.

### 9. Какие четыре состояния файлов существуют в Git?

Untracked (неотслеживаемый), Modified (изменённый), Staged (подготовленный), Committed (закоммиченный).

**10. В чём разница между состояниями untracked и modified?**

Untracked — это новый файл, который Git ещё не отслеживает. Modified — это файл, который уже был в репозитории, но в него были внесены изменения.

**11. Чем отличается команда git add . от git add -A и git add <имя файла>?**

git add . добавляет все файлы из текущей папки и вложенных. git add -A добавляет все файлы во всём репозитории (включая удалённые). git add <имя файла> добавляет только конкретный файл.

**12. Что такое коммит и какие метаданные он содержит?**

Коммит — это основной объект в Git, содержащий изменения. Он содержит: уникальный идентификатор (хеш), имя автора, дату создания и комментарий с описанием изменений.

**13. Для чего нужна команда git status и какую информацию она отображает?**

Показывает текущее состояние файлов в репозитории: какие файлы изменены, какие подготовлены к коммиту, а какие ещё не отслеживаются.

**14. Что показывает команда git diff и в каких случаях она применяется?**

Показывает разницу между текущими изменениями в файлах и последним коммитом. Применяется для просмотра того, что именно было изменено перед добавлением в индекс.

**15. Для чего используется файл .gitignore и как формируются его шаблоны?**

Файл .gitignore используется для игнорирования определённых файлов и папок (например, скомпилированных классов, временных файлов). Каждая строка в нём — это шаблон (например, \*.class, target/, .idea/).

**16. Что такое ветка в Git и почему её называют подвижным указателем?**

Ветка — это указатель на определённый коммит. При создании нового коммита ветка автоматически переносится на него, поэтому её называют подвижной.

**17. В чём разница между командами git checkout master и git checkout -b new-branch?**

git checkout master переключает на существующую ветку master. git checkout -b new-branch создаёт новую ветку и сразу переключается на неё.

**18. Что происходит при выполнении команды git merge и в каком случае возникает конфликт?**

Команда git merge выполняет слияние одной ветки в другую. Конфликт возникает, если в двух ветках были изменены одни и те же строки одного и того же файла, и Git не может автоматически определить, какую версию оставить.

**19. Как разрешить конфликт слияния и какие маркеры при этом используются в файле?**

Конфликт разрешается вручную: нужно открыть файл, удалить маркеры конфликта (<<<<<<< HEAD, =====, >>>>>>> branch\_name), оставить нужный код, затем выполнить git add и git commit.

**20. Для чего применяется команда git log --oneline --graph --all?**

Она показывает историю коммитов в сокращённом виде (`--oneline`), в виде графа (`--graph`), включая все ветки (`--all`). Это удобно для визуального анализа структуры ветвления.

## Вывод

В ходе выполнения лабораторной работы были получены практические навыки работы с распределённой системой контроля версий Git в операционной системе Linux.

В рамках работы была проверена наличие Git в системе командой `git --version` и определён дистрибутив Linux с помощью `cat /etc/os-release`. При необходимости Git был установлен через менеджер пакетов. Настроены имя пользователя и адрес электронной почты командами `git config --global user.name` и `git config --global user.email`. Создана рабочая директория `git-practice`, в которой инициализирован локальный репозиторий командой `git init`, и проверено создание скрытой папки `.git`.

В домашней директории создано несколько файлов различных типов (`readme.txt`, `script.sh`, `data.json`) с помощью команды `touch`. Изучены состояния файлов в Git: `untracked`, `modified`, `staged`, `committed`. Выполнено добавление файлов в индекс командами `git add *.txt` и `git add -A`, создан первый коммит командой `git commit -m "initial commit: added base files"`. Просмотрена история коммитов с помощью `git log` и зафиксированы метаданные коммита.

Внесены изменения в файл `readme.txt` и просмотрены различия командой `git diff`. Создан файл `.gitignore` с шаблонами `*.log`, `temp/`, `*.tmp`, `.cache/` для игнорирования служебных файлов. Освоено создание веток командой `git checkout -b feature/update-readme`, просмотр списка веток командой `git branch -a`, внесение изменений в новой ветке, переключение на `master` командой `git checkout master` и слияние командой `git merge feature/update-readme`.

Проанализирована итоговая история командой `git log --oneline --graph --all`.

Таким образом, цель работы достигнута — освоены базовые и продвинутые приёмы работы с системой контроля версий Git, включая установку, настройку, управление состояниями файлов, работу с ветками, слияние и разрешение конфликтов, что формирует практические навыки организации версионного контроля при разработке программного обеспечения.